

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Debugging Test

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 3 – Debugging Test
Weightage:		Total Marks:	25 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	Dharshini H	Reg. No.:	192324268
Section / Batch:	B.Tech AI & DS	Date of Submission:	04-08-2026
Faculty In-charge:	Dr. B.SHYAMALA GOWRI	Project Mode:	Individual Submission

Code of Conduct

I Dharshini H 192324268 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

Error Identification(5)	Code Correction(8)	SQL Query Accuracy(5)	Explanation and Justification(4)	Program Execution and Output(3)	Total (25 Marks)

Q1. Debug the SQLite Program The following program is intended to create a table and insert student records. However, it contains multiple errors.

```
import sqlite3
conn = sqlite3.connect(college.db)
cursor = conn.cursor
cursor.execute("""
CREATE TABLE Student(
id INTEGER PRIMARY KEY
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES(1,'John',90)")
conn.close()
```

Task:

1. Identify **at least five errors**.
2. Rewrite the corrected program.
3. Explain the purpose of each correction.

Error	Incorrect Code	Correct Code	Purpose of Correction
1	sqlite3.connect(college.db)	sqlite3.connect("college.db")	Database name must be a string enclosed in quotes.
2	cursor = conn.cursor	cursor = conn.cursor()	cursor() is a method and must be called using parentheses.
3	Missing comma after id INTEGER PRIMARY KEY	id INTEGER PRIMARY KEY,	Columns in a SQL table definition must be separated by commas.
4	No conn.commit() after inserting data	conn.commit()	Saves the inserted record permanently in the database.
5	Connection closed without committing changes	Commit before conn.close()	Ensures all changes are written before closing the database.

```

1  import sqlite3
2  conn = sqlite3.connect("college.db")
3  cursor = conn.cursor()
4  cursor.execute("""
5  CREATE TABLE Student(
6      id INTEGER PRIMARY KEY,
7      name TEXT,
8      marks INTEGER
9  )
10 """)
11 cursor.execute("INSERT INTO Student VALUES (1, 'John', 90)")
12 conn.commit()
13 conn.close()

```

Q6. Debug the Transaction Program

```

import sqlite3
conn=sqlite3.connect("college.db")cursor=conn.cursor()
cursor.execute("UPDATE Student SET marks=80 WHERE id=2")
cursor.execute("INSERT INTO Student VALUES(2,'John',80)")
conn.rollback()
conn.commit()
conn.close()

```

Task

- Identify logical errors related to transactions.
- Explain the effect of rollback() and commit().
- Rewrite the corrected program.

1. Identify the Logical Errors

Error	Explanation
1. Missing newline/statement separator	conn=sqlite3.connect("college.db")cursor=conn.cursor() is invalid Python syntax. These must be on separate lines.
2. rollback() is called before commit()	rollback() cancels all uncommitted changes, so the UPDATE and INSERT are undone. Calling commit() afterward has nothing to save.
3. Updating before checking if the record exists	UPDATE Student SET marks=80 WHERE id=2 has no effect if no student with id=2 exists.
4. Possible duplicate primary key	INSERT INTO Student VALUES(2,'John',80) will fail if id=2 already exists because id is the primary key.

2. Effect of rollback() and commit()

- **commit()**
 - Saves all changes made during the current transaction permanently to the database.
 - After a commit, the changes cannot be undone using rollback().
- **rollback()**
 - Cancels all changes made since the last commit().
 - Restores the database to its previous committed state.
 - It is mainly used when an error occurs during a transaction.

Corrected Program

CO2 >  AT2 > ...

```
3  import sqlite3
4  # Connect to the database
5  conn = sqlite3.connect("college.db")
6  cursor = conn.cursor()
7  try:
8      # Update marks of student with id = 2
9      cursor.execute("UPDATE Student SET marks = 80 WHERE id = 2")
10     # Commit the transaction
11     conn.commit()
12     print("Transaction completed successfully.")
13 except Exception as e:
14     # Undo changes if an error occurs
15     conn.rollback()
16     print("Transaction failed:", e)
17 finally:
18     # Close the connection
19     conn.close()
```